Appendix B: Architecture Review Template and Question Bank

> This appendix gives you a one-page architecture review template and a question bank of 99 questions grouped by topic. Use the template to frame a review and the questions to probe a proposal. You do not need to ask every question. Pick the ones that match the risk of the system: a patient-facing GenAI assistant needs far more scrutiny on compliance and evaluation than an internal territory-planning dashboard. Terms such as HCP, HCO, MLR, and RAG are defined in Appendix A.

Part 1: The one-page review template

Copy this structure into the review ticket or document. Keep each section short. If a section cannot be filled in, that gap is itself a finding.

Field	Entry
System name and version	Name the system and the release or design version under review
Business owner and data steward	The accountable business lead and the steward for each data domain used
Engineering owner	The team that will build and run the system
Reviewers	Architecture, security, privacy, compliance (GxP or MLR where relevant), and data governance
Review type and date	Design review, pre-production, or post-incident; date of the session
Decision requested	Approve, approve with conditions, or return for rework
Regulatory classification	Non-GxP, GxP-validated, patient-facing, promotional, or medical; any PHI or PII involved

1. Summary and business problem

State the decision or workflow the system supports, who uses it, and what happens if it is late, wrong, or unavailable. Two to four sentences is enough.

2. Requirements

- **Functional**: the capabilities the system must have, written as user-visible behavior.

- **Non-functional**: latency, freshness, availability, throughput, retention, and recovery targets, each with a number.
- **Out of scope**: what the system will explicitly not do in this release.

3. Users and consumers

List each user group and each downstream system that reads the outputs, with the access method and expected volume.

4. Data

- Sources, owners, licenses, and contractual restrictions (for example, Xponent, claims, CRM, or internal medical content).
- Classification (PHI, PII, confidential, internal, public) and the handling rule for each.
- Volumes, refresh cadence, and the freshness commitment.
- Lineage from source to consumer, and where data quality rules run.

5. Architecture

Include one diagram showing components, data flows, trust boundaries, and where humans approve actions. Add a table with each component, its technology, its owner, and whether it is managed or self-hosted. Mermaid or ASCII diagrams are both acceptable.

6. Compute, scale, and resilience

Describe peak load and the design for it, the failure modes considered, the redundancy and failover approach, and the recovery point and recovery time objectives (RPO and RTO).

7. Security and compliance

Describe identity and access (including row-level or attribute-based controls), secrets handling, encryption, network boundaries, audit logging, retention, and the validation evidence required for any GxP component.

8. AI-specific design (if applicable)

- Model or models used, whether vendor-hosted or self-hosted, and the version pinning policy.
- Grounding and retrieval approach, and how citations are produced.
- Evaluation plan: test sets, metrics, thresholds for release, and the regression suite.
- Guardrails, human-in-the-loop points, and how prompt injection and PII leakage are controlled.
- Monitoring for drift, quality, and cost, and the rollback path for a model or prompt change.

9. Cost

Estimate run cost per month at expected and peak volume, the main cost drivers (compute, tokens, storage, licenses), and the budget owner.

10. Operations

On-call ownership, alerting thresholds, runbooks for the top five failure modes, release process, and the support model for business users.

11. Risks, open questions, and decisions

#	Risk or question	Severity	Owner	Mitigation or decision	Due date

12. Review outcome

- Decision: approve, approve with conditions, or return for rework.
- Conditions with named owners and dates.
- Review follow-up date and the criteria for closing the review.
- Sign-off by business owner, engineering owner, and each required function.

Part 2: Question bank

Questions are grouped by topic. Each one is answerable with yes, no, or a short explanation, and each one points at a specific failure mode. Use them in design reviews, in intake meetings, and when a vendor proposal arrives.

Requirements

1. Can the business owner state the decision or workflow this system improves in one sentence, with a measurable outcome?
2. What happens to the business if the system is wrong, late, or unavailable for one day, one week, and one month?
3. Are the freshness, latency, and availability targets written as numbers, and who agreed to them?
4. Which requirements are must-have for launch and which are deferred, and is the deferral recorded?
5. Has anyone compared this design with a simpler baseline, such as a rules-based approach or a manual process?
6. Who are the users, how many are there at launch and at the one-year mark, and what devices and networks do they use?

7. What does success look like in the first 90 days, and how will it be measured with data the business trusts?
8. Which regulatory or contractual obligations apply, and has the compliance function confirmed the classification?
9. What is explicitly out of scope, and how will requests to expand scope be handled?
10. If the project were cancelled tomorrow, what would have been learned and what would be reusable?

Data

1. Which source systems feed this design, and does each one have a named owner and a documented license or contract that permits this use?
2. Is there a data dictionary and source-to-target mapping for every table the system reads or writes?
3. What is the freshness commitment for each source, and what happens when a source is late?
4. How are duplicate records across sources resolved, and what is the expected match rate for HCP and HCO entities?
5. Which data quality rules run before data reaches consumers, and what happens when a rule fails?
6. Is there lineage from each output back to source records, and can an analyst trace a single number in under 15 minutes?
7. How does the design handle schema drift when a vendor adds, renames, or retypes a field?
8. Are historical corrections handled as new records or as overwrites, and can past reports be reproduced as they were?
9. Where does personal or protected data appear in logs, prompts, caches, and exports, not just in the main tables?
10. How much data is expected in year one and year three, and does the storage and compute design still hold at those volumes?

Compute

1. What is the expected peak load, when does it occur, and is that peak planned or accidental?
2. Which workloads share compute, and could a heavy batch job slow down the morning dashboards?
3. How long does a cold start take for each service or model, and does the user experience tolerate it?
4. Is the design sized from measured or estimated throughput, and what are the assumptions behind the estimate?
5. Can compute scale up or down automatically, and has anyone tested the scaling behavior under load?
6. Which components are stateful, and how are they scaled and recovered without losing data?

7. Are batch jobs idempotent, so that rerunning a failed job does not double count?
8. Where are the GPU or other specialized resources, who owns capacity planning, and what is the lead time for more?
9. What is the plan for running this design in a second region or environment, if one is required?
10. Could a single query or prompt consume the entire budget for a team or project, and what quota prevents that?

Reliability

1. What is the availability target for each user-facing function, and what is the measured history for comparable systems?
2. What are the top five ways this system can fail, and for each one, what does the user see?
3. Which single points of failure remain, and is that acceptable for the business impact?
4. Are timeouts, retries, and circuit breakers defined for every external dependency, including vendor APIs?
5. What are the recovery point and recovery time objectives, and has a restore been tested in the last six months?
6. How do failed records get captured, reprocessed, and reported, rather than silently dropped?
7. What happens to in-flight work during a deployment or failover?
8. Is there a rollback plan for each release type, including data migrations and model or prompt changes?
9. How do upstream outages, such as a late vendor file, propagate into downstream dashboards and campaign decisions?
10. What evidence shows that disaster recovery has been exercised, and who signs off that the result was acceptable?

Security and compliance

1. How do users authenticate, and is multi-factor authentication enforced for all access to protected or confidential data?
2. What is the authorization model, and can it limit a rep to their own territory or a market access analyst to their assigned products?
3. Where are secrets stored, who can read them, and how are they rotated?
4. Is data encrypted at rest and in transit, and where are the encryption keys managed?
5. Which network boundaries exist, and can any component reach the internet that does not need to?
6. Is every access to protected data and every change to configuration recorded in an audit trail that users cannot edit?
7. For any GxP component, is there a validation plan, a traceability matrix, and documented evidence of testing?
8. How long is each class of data retained, and is deletion automated and verified?

9. Has a privacy review classified the data as de-identified, limited, or identifiable, and does that classification match the design?
10. Are third-party vendors assessed for security and for data processing terms, and do the contracts cover the data flows described here?

AI-specific

1. What is the business task the model performs, and is it predictive, generative, or a combination?
2. Which model or models are used, are they pinned to a version, and what happens when the vendor retires that version?
3. What evaluation set represents real usage, who labeled it, and how often is it refreshed?
4. What quality threshold must the system meet to launch, and what happens if it falls below that threshold in production?
5. For retrieval systems, what fraction of answers cite a source that actually supports the claim, and how was that measured?
6. What guardrails block unsafe, off-label, or unsupported output, and how often do they fire in testing?
7. Where is a human reviewer required before an output takes effect, and how is that reviewer's decision recorded?
8. How is prompt injection from retrieved documents or user input tested and contained?
9. Are model inputs and outputs logged, and what is the retention and access policy for those logs?
10. How will the system detect data drift, concept drift, or a drop in answer quality after launch, and who is alerted?
11. Can the business explain why a recommendation was made, at the level the risk classification requires?
12. Has the team tested whether the model performs differently across customer segments, geographies, or practice types?
13. What is the rollback path for a prompt, model, or embedding change, and has it been rehearsed?
14. If the system uses a semantic cache, how is a wrong match detected and prevented?

Cost

1. What is the expected monthly run cost at launch, at expected growth, and at peak, with the assumptions written down?
2. Which line items drive most of the cost, and is each one attributed to a budget owner?
3. What is the cost per unit of business value, such as per call plan generated, per approved content piece, or per answered question?
4. Is there a model tiering or routing strategy that sends simple requests to cheaper models?

5. Are repeated computations or retrievals cached where the freshness requirement allows it?
6. Are development, test, and training environments sized and scheduled to avoid running 24 hours a day?
7. What is the cost of storing raw, intermediate, and historical copies of data, and is there a retention policy for each?
8. Are there budget alerts, and what action is taken automatically when spend exceeds the forecast?
9. What is the cost of a 10 percent increase in volume, and does the pricing model of the vendor scale in a way the business can accept?

Operations

1. Who is on call for this system, what hours do they cover, and what is the escalation path to the business owner?
2. Which metrics and logs are monitored, and which alert thresholds are tied to the service level objectives?
3. Is there a runbook for each of the top five failure modes that a new engineer could follow without help?
4. How are releases promoted from development to test to production, and who approves each promotion?
5. How are changes to data pipelines, prompts, and models tracked and reviewed, and can a past version be reproduced?
6. How do business users report problems and get them triaged, and what is the target time to first response?
7. What training and documentation do data stewards and business users need, and who maintains it after launch?
8. What is the plan for decommissioning this system, including data archival and the handling of dependent reports?
9. How will the team measure whether the system is still delivering value six and twelve months after launch?
10. What operational work will be needed each week, each month, and each quarter, and has the team staffed it?

Enterprise agent infrastructure

These questions apply when agents reach enterprise systems through tools, for example over the Model Context Protocol (MCP), or talk to other agents. Chapters 18 and 19 explain the concepts, and Appendix C shows working examples.

1. Do all agent tool calls pass through an MCP gateway (or equivalent control point), or can any agent reach an MCP server or backend system directly?

2. Does the gateway enforce per-role and per-agent tool allow-lists, rate limits, redaction of sensitive data in tool results, and an audit record of every call with user, agent, tool, and arguments?
3. Is there an approved-server registry with a named owner for each MCP server, and are approved tool descriptions pinned (for example by hash) so that a changed description is blocked until re-reviewed?
4. Has every third-party MCP server been reviewed for tool poisoning (hidden instructions in descriptions), and is there a process for the "rug pull" case where a server changes after approval?
5. Does each agent have its own identity in the identity provider, separate from human users and from shared service accounts, with a named owner and a review date?
6. When an agent acts for a user, does it obtain a short-lived, downscoped token through token exchange, with the correct audience for each server and an act claim naming the agent, rather than using a broad service account?
7. Do backend tools apply the end user's entitlements (caller's rights, row access policies), so the agent cannot become a confused deputy that returns data the user may not see?
8. Are tool annotations (read-only, destructive) accurate, and is read-only behavior enforced by grants and policy rather than by trusting the annotation?
9. Is the MCP transport appropriate: stdio only for local single-user tools, and Streamable HTTP with authentication and network controls for shared servers?
10. If agents call other agents (for example over A2A), are agent cards published only for approved agents, is each call authenticated, and does the user's identity and consent carry across the hop?
11. Is there a kill switch that can disable a single tool, a single MCP server, or a single agent within minutes, and has it been rehearsed?
12. Does every agent task produce one trace, with a span for each model call and tool call carrying tokens, cost, latency, and guardrail outcomes, in a standard format such as OpenTelemetry?
13. Are tools, prompts, and agent configurations versioned with semantic versioning, and does an automated check flag breaking changes to tool schemas before release?
14. Are dev, test, and prod environments for the agent platform defined as infrastructure as code (for example Terraform), with access rules expressed as policy as code?
15. Does promotion from dev to test to prod require a passing eval gate, including red-team probes for injection through tool results, with the run ID recorded for each release?
16. Is cost attributed per agent, per task, and per cost center through the LLM gateway, with step and token budgets per task enforced outside the model?

Part 3: Using the review

Run the review in two passes. In the first pass, the presenting team walks through the template and answers the questions that apply to the system's risk level. In the second pass, reviewers challenge

the answers and the team records each gap in the risks table with an owner and a due date. A review that ends with no open risks has usually not looked hard enough. A review that ends with dozens of open risks usually has the wrong scope, and the system should be split into smaller releases.

Record the decision in the review ticket and attach the completed template. Revisit the review when the design changes materially, when the first production incident occurs, or when the business owner changes.